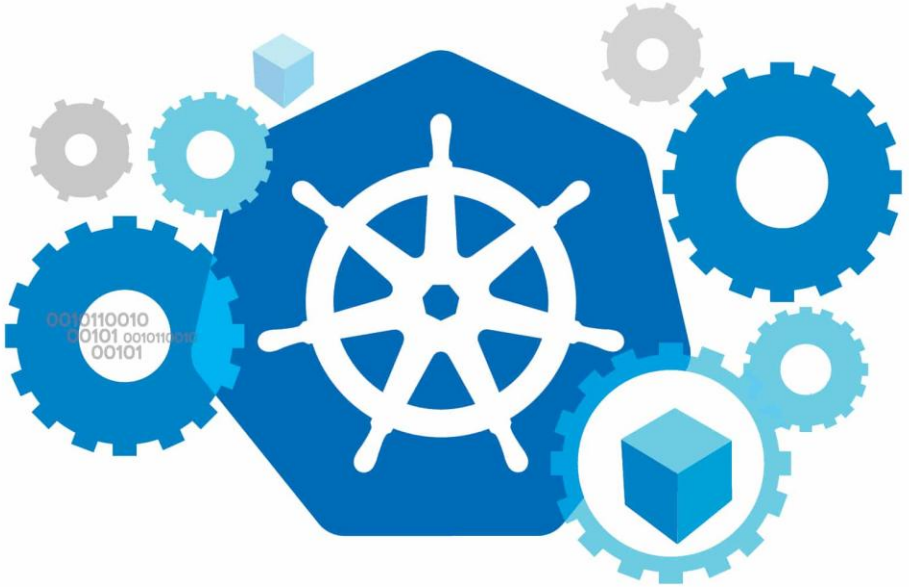


INGRESS X GATEWAY API

A escolha da melhor opção para seu ecossistema



Igor Gouvea

DevOps Engineer - A4PM

Graduando em Cibersegurança

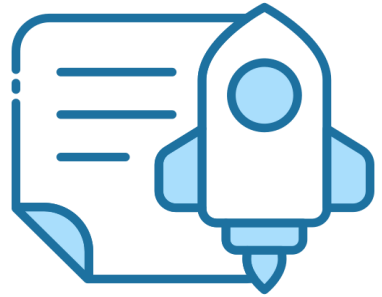






Do container a exposição

- Pods: Onde tudo começa
- O desafio da comunicação entre componentes
- Services: Solução para uma comunicação estável
- O problema de custos e escala



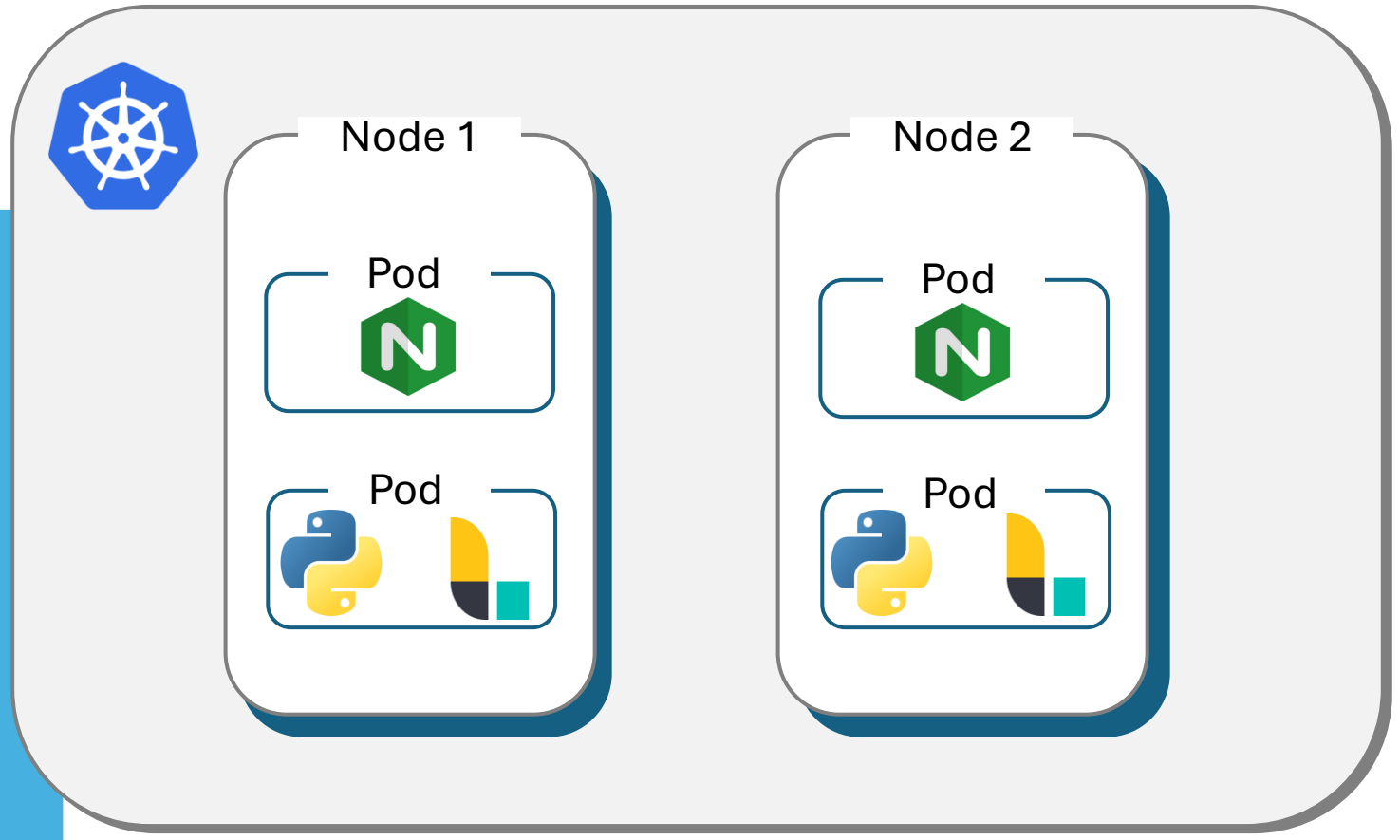
Evoluindo a camada de entrada

- Ingress: Uma solução mais inteligente
- Novo desafio: Escala organizacional
- Limitações técnicas do Ingress
- Gateway API: A evolução do roteamento do Kubernetes

PODS



- É a menor unidade de implantação do Kubernetes
- Local onde nossos containers são executados
- Considerado como um host isolado para os containers

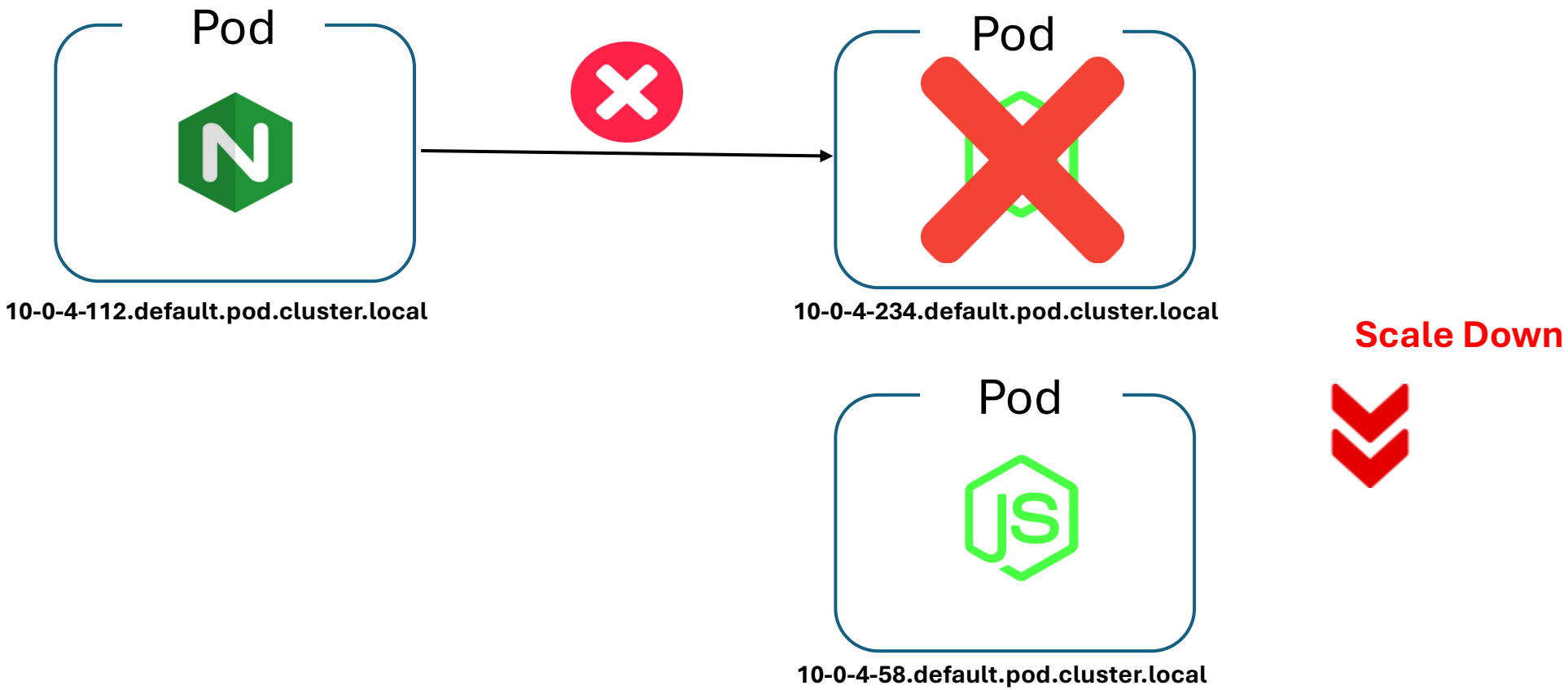


```yaml
apiVersion: v1
kind: Pod
metadata:
  name: backend-api
  labels:
    app: api
spec:
  containers:
  - name: api
    image: node:22
    ports:
    - containerPort: 8080
    resources:
      requests:
        cpu: 500m
        memory: 64Mi
      limits:
        cpu: 2000m
        memory: 512Mi
```

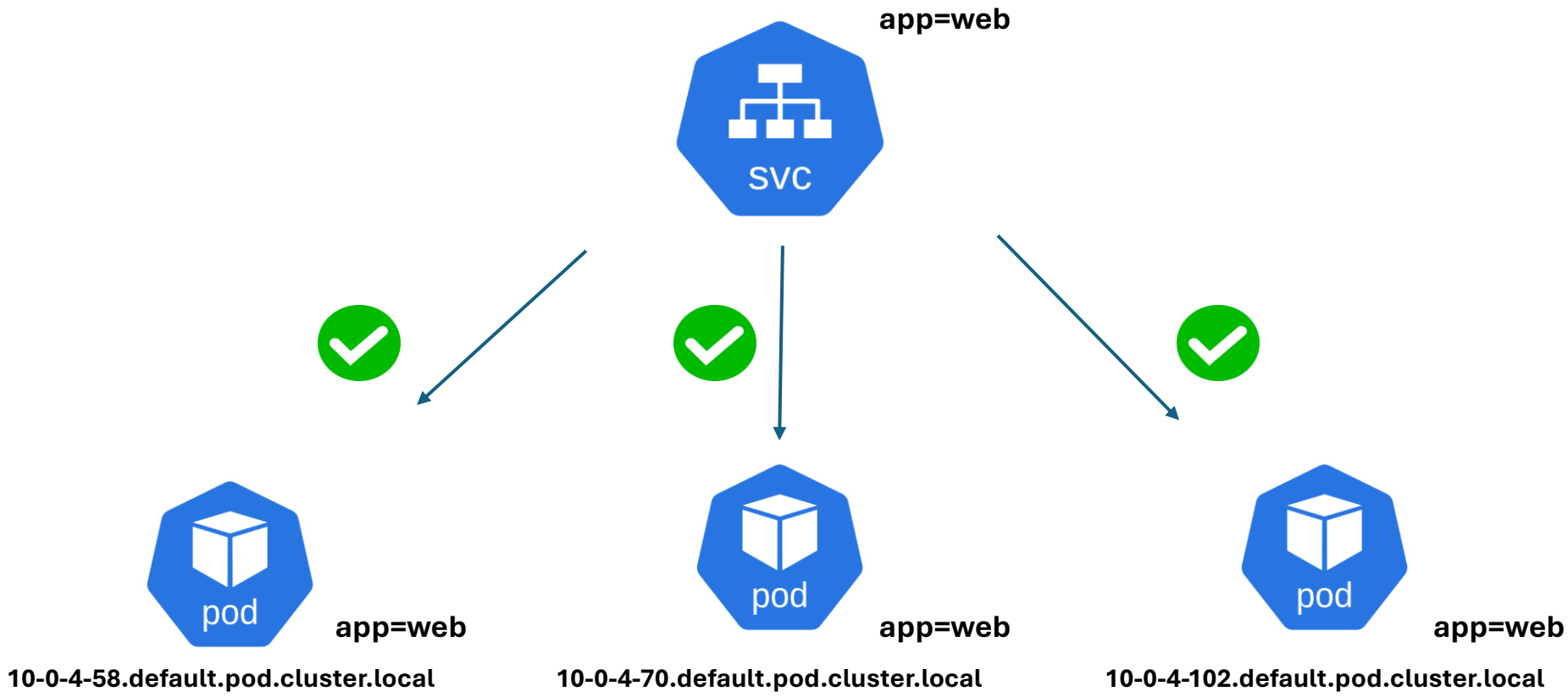
```yaml
apiVersion: v1
kind: Pod
metadata:
  name: backend-admin
  labels:
    app: admin
spec:
  containers:
  - name: admin
    image: node:22
    ports:
    - containerPort: 8080
    resources:
      requests:
        cpu: 500m
        memory: 64Mi
      limits:
        cpu: 2000m
        memory: 512Mi
```

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: backend-auth
  labels:
    app: auth
spec:
  containers:
  - name: auth
    image: node:22
    ports:
    - containerPort: 8080
    resources:
      requests:
        cpu: 1000m
        memory: 64Mi
      limits:
        cpu: 3000m
        memory: 1024Mi
```

COMUNICAÇÃO ENTRE COMPONENTES



SERVICES


app=web
svc
pod
app=web
10-0-4-58.default.pod.cluster.local
pod
app=web
10-0-4-70.default.pod.cluster.local
pod
app=web
10-0-4-102.default.pod.cluster.local

Tipos de services

- ClusterIP
- NodePort
- LoadBalancer (Usado em produção)

```yaml
apiVersion: v1
kind: Service
metadata:
  name: backend-api-svc
spec:
  type: ClusterIP
  selector:
    app: api
  ports:
  - protocol: TCP
    port: 80
    targetPort: 8080
```

```yaml
apiVersion: v1
kind: Service
metadata:
  name: backend-admin-svc
spec:
  type: ClusterIP
  selector:
    app: admin
  ports:
  - protocol: TCP
    port: 80
    targetPort: 8080
```

```yaml
apiVersion: v1
kind: Service
metadata:
  name: backend-auth-svc
spec:
  type: ClusterIP
  selector:
    app: auth
  ports:
  - protocol: TCP
    port: 80
    targetPort: 8080
```

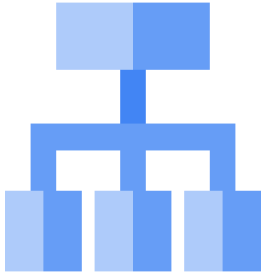
Barreiras ao expor apps com vários LoadBalancers



Custo elevado na exposição de serviços



Aumento na sobrecarga operacional



INGRESS

- Gerencia o acesso externo aos serviços do cluster
- Trabalha na camada 7 do modelo OSI
- Suporte nativo para roteamento de protocolos HTTP/S
- Trabalha em conjunto com um Ingress Controller

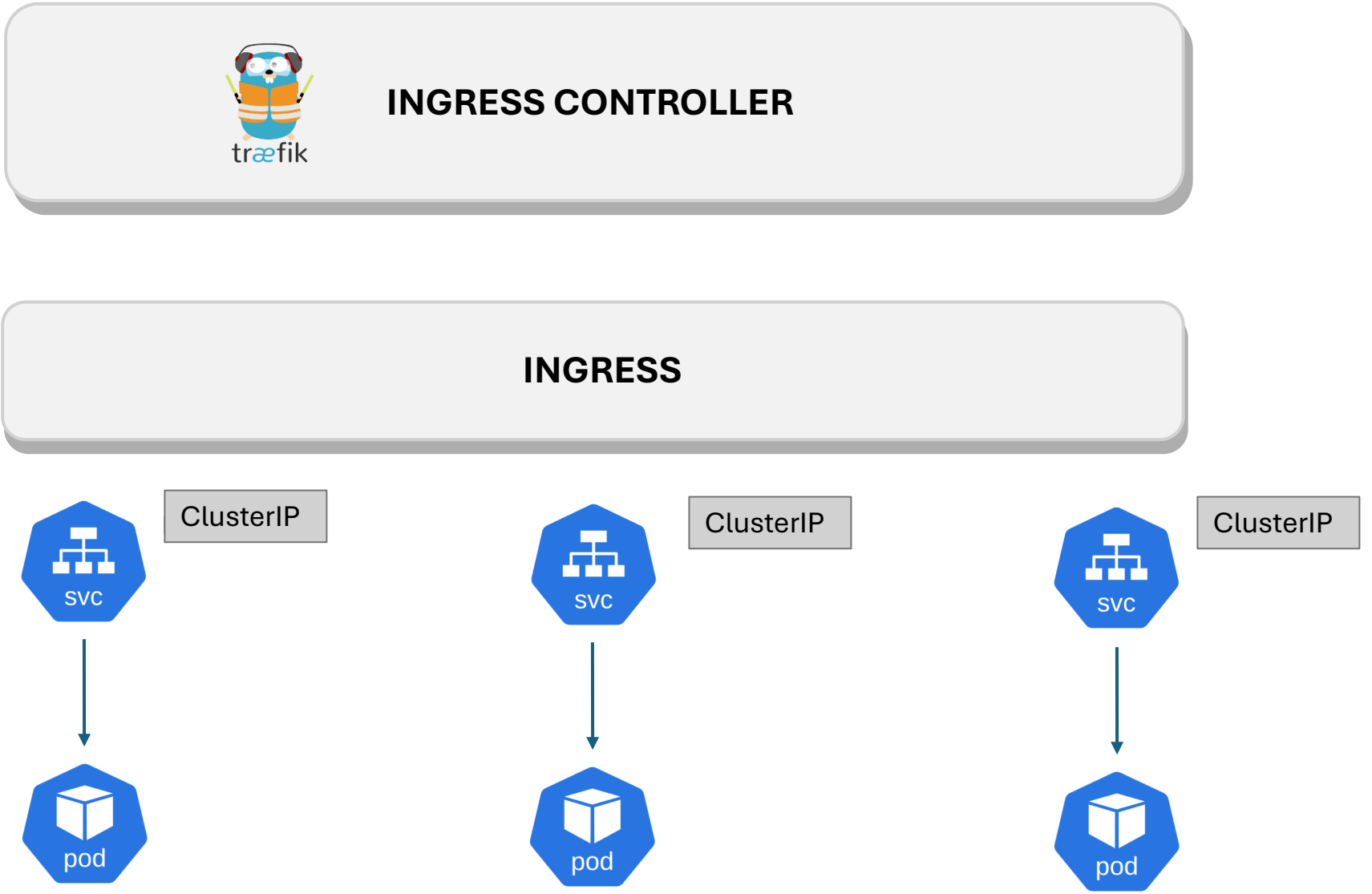
Alguns exemplos de Ingress Controller disponíveis










træfik
INGRESS CONTROLLER
INGRESS
ClusterIP
svc
pod
ClusterIP
svc
pod
ClusterIP
svc
pod



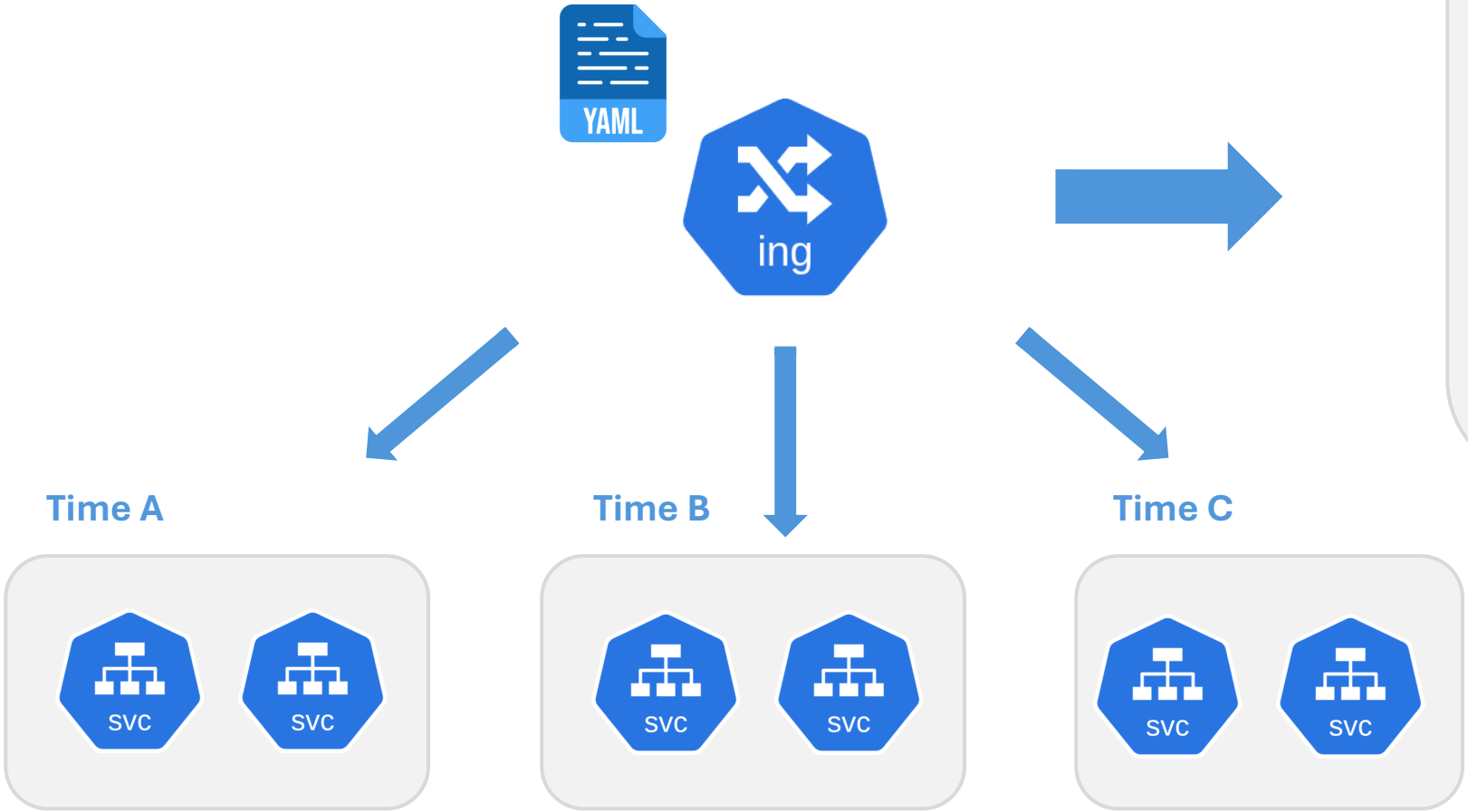


Exemplo de Ingress




```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: backend-ingress
  annotations:
    nginx.ingress.kubernetes.io/force-ssl-redirect: "true"
spec:
  ingressClassName: nginx
  tls:
  - hosts:
    - backend.meusite.com
    secretName: meusite-tls
  rules:
  - host: backend.meusite.com  # matchHost
    http:
      paths:
      - pathType: Prefix
        path: "/"       # matchPath
        backend:
          service:
            name: backend-api-svc
            port:
              number: 80
      - pathType: Prefix
        path: "/admin"  # matchPath
        backend:
          service:
            name: backend-admin-svc
            port:
              number: 80
      - pathType: Prefix
        path: "/auth"   # matchPath
        backend:
          service:
            name: backend-auth-svc
            port:
              number: 80
```

NOVOS DESAFIOS



Sem suporte nativo

- Redirecionamento https
- Manipulação de header
- Limite de requisições
- Sistema de autenticação

ANNOTATIONS



```yaml
# Traffic Splitting

metadata:
  annotations:
    nginx.ingress.kubernetes.io/canary: "true"
    nginx.ingress.kubernetes.io/canary-weight: "20"
---
# SSL Redirect

metadata:
  annotations:
    nginx.ingress.kubernetes.io/force-ssl-redirect: "true"
---
# Header Manipulation

annotations:
  nginx.ingress.kubernetes.io/configuration-snippet: |
    more_set_headers "X-Custom-Header: value";
    more_set_headers "X-Frame-Options: DENY";
  nginx.ingress.kubernetes.io/proxy-set-headers: "custom-headers"
```




```yaml
# Traffic Splitting

metadata:
  annotations:
    haproxy.org/server-weight: "20"
---
# SSL Redirect

metadata:
  annotations:
    haproxy.org/ssl-redirect: "true"
---
# Header Manipulation

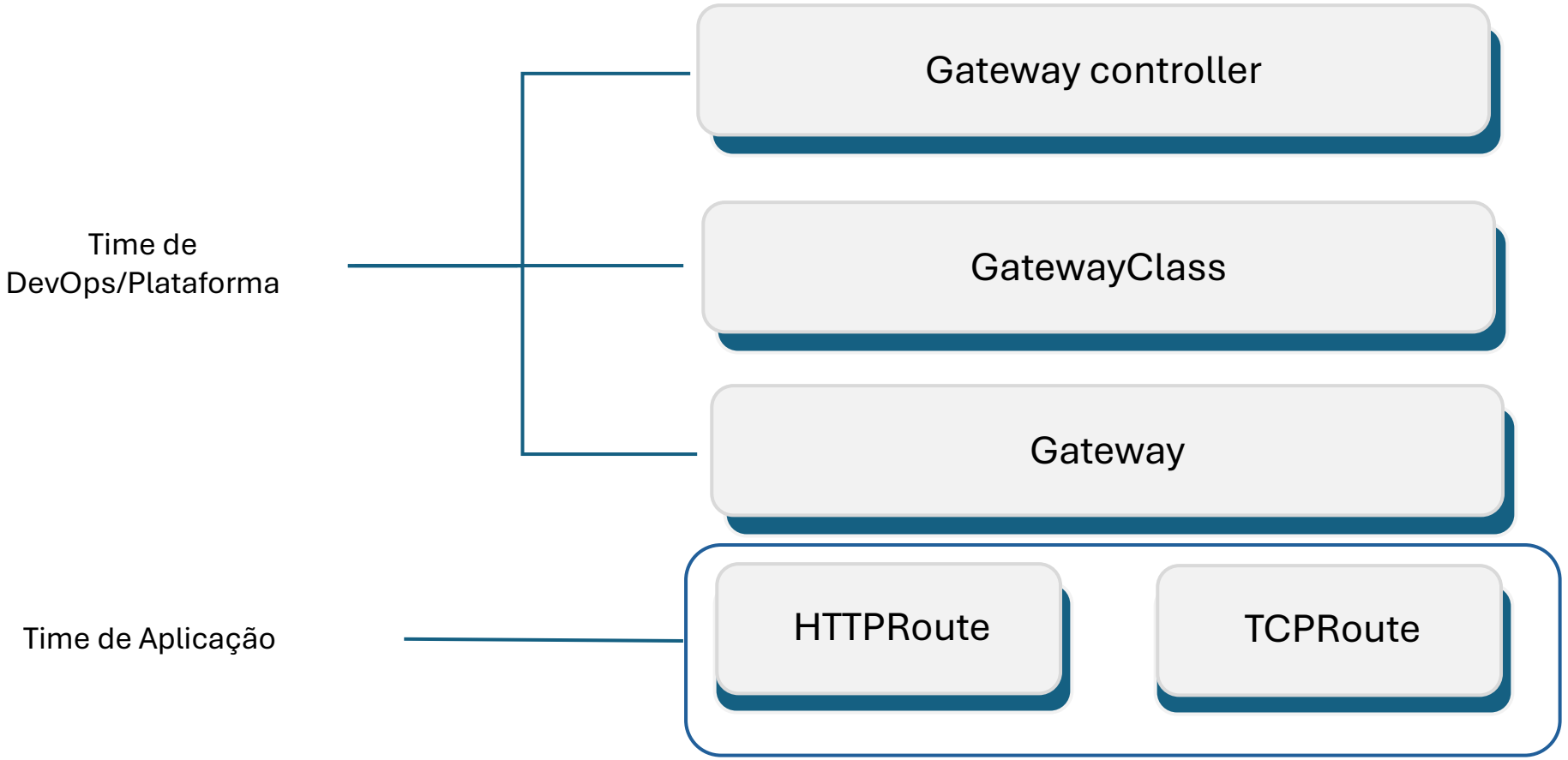
annotations:
  haproxy.org/request-set-header: |
    X-Custom-Header value
  haproxy.org/response-set-header: |
    X-Frame-Options DENY
```

GATEWAY API

- Uma evolução do fluxo de roteamento do kubernetes
- Trabalha na camada 4 e7 do modelo OSI
- Suporte nativo a roteamentos HTTP/S, TCP, UDP e gRPC
- Estrutura modularizada
- Regras avançadas de tratamento de requisições
- Permite divisão de responsabilidades



Estrutura Básica



```yaml
apiVersion: gateway.networking.k8s.io/v1
kind: GatewayClass
metadata:
  name: nginx
spec:
  controllerName: gateway.nginx.org/nginx-gateway-controller
  description: "NGINX Gateway Fabric GatewayClass"
```




```yaml
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  name: backend-gateway
spec:
  gatewayClassName: nginx
  listeners:
  - name: http
    protocol: HTTP
    port: 80
    hostname: backend.meusite.com
    allowedRoutes:
      namespaces:
        from: Same
    rules:
    - filters:
      - type: RequestRedirect
        RequestRedirect:
          scheme: https
          statusCode: 301
          port: 443
  - name: https
    protocol: HTTPS
    port: 443
    hostname: backend.meusite.com
    allowedRoutes:
      namespaces:
        from: Same
    tls:
      mode: Terminate
      certificateRefs:
      - name: meusite-tls
```

```yaml
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: backend-routes
spec:
  parentRefs:
  - name: backend-gateway
  hostnames:
  - backend.meusite.com
  rules:
  - matches:
    - path:
        type: PathPrefix
        value: /
    backendRefs:
    - name: backend-api-svc
      port: 8080

  - matches:
    - path:
        type: PathPrefix
        value: /admin
    backendRefs:
    - name: backend-admin-svc
      port: 8080

  - matches:
    - path:
        type: PathPrefix
        value: /auth
    backendRefs:
    - name: backend-auth-svc
      port: 8080
```

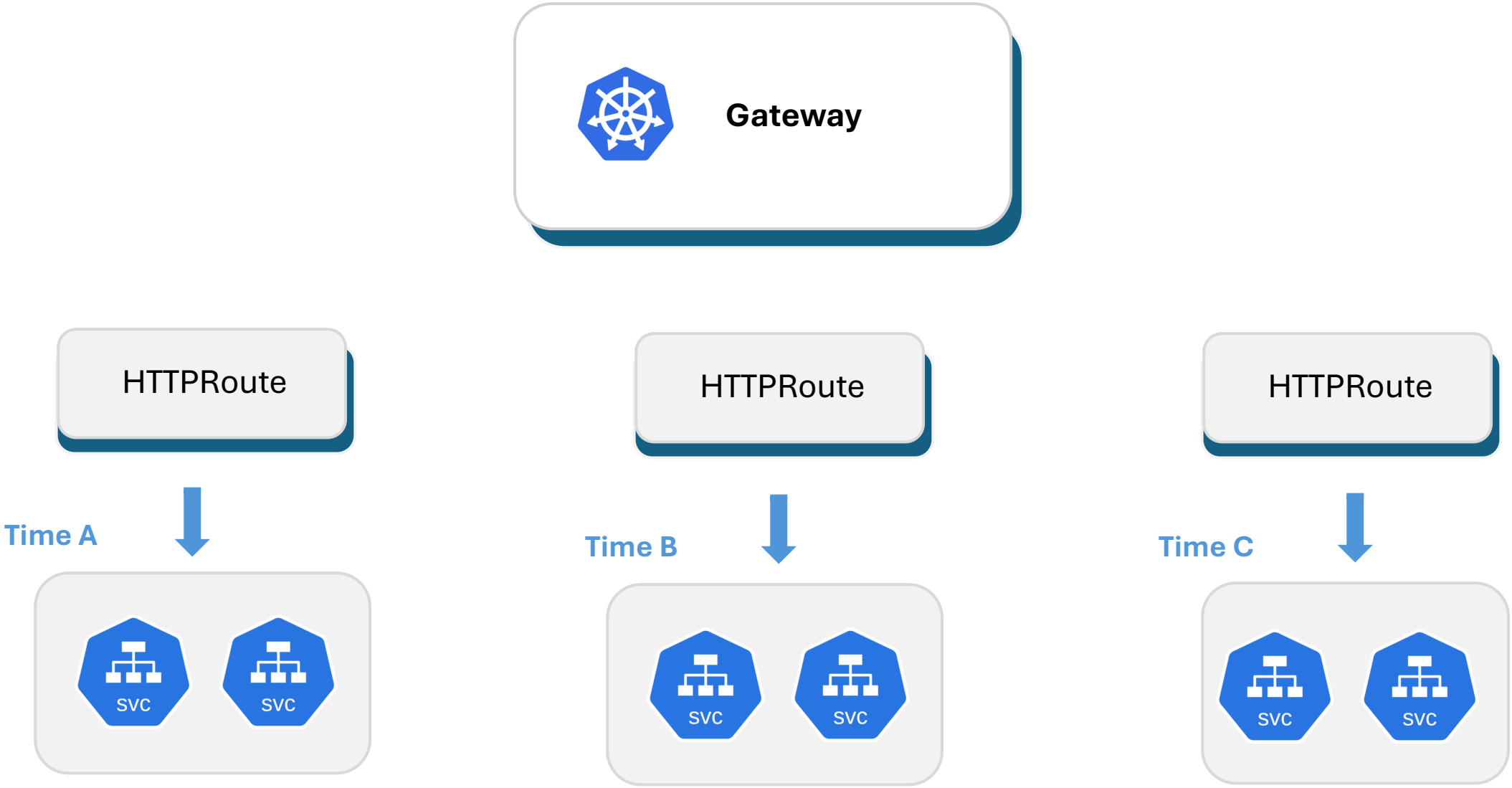
Configurações avançadas de roteamento

```yaml
# Traffic Splitting
spec:
  rules:
  - backendRefs:
    - name: service-1
      port: 80
      weight: 90
    - name: service-2
      port: 80
      weight: 10
```

```yaml
# Header Manipulation
spec:
  rules:
  - filters:
    - type: RequestHeaderModifier
      requestHeaderModifier:
        add:
        - name: X-Custom-Header
          value: custom-value
```

```yaml
# Rate Limit
spec:
  targetRef:
    group: gateway.networking.k8s.io
    kind: HTTPRoute
    name: backend-route
  rateLimit:
    type: Local
    local:
      rules:
      - limit:
          requests: 100
```

NOVO MODELO


Gateway
HTTPRoute
HTTPRoute
HTTPRoute
Time A
Time B
Time C
SVC
SVC
SVC
SVC
SVC
SVC

Quando utilizar cada recurso



Ingress

- Ambientes simples e pequenos
- Equipes centralizadas
- Casos de roteamento básicos
- Stack já estabelecida



Gateway API

- Atuação com múltiplos times
- Separação de responsabilidades
- Roteamento para múltiplos protocolos
- Necessidade de roteamento avançado
- Arquiteturas que exigem escala

OBRIGADO!!